



Birla Institute of Technology & Science, Pilani Pilani Campus

ACADEMIC— UNDERGRADUATE STUDIES DIVISION

Semester 2026-27 Course Handout Part II

Date 04-08-2026

In addition to part I (General Handout for all courses appended to the Timetable), this portion gives further specific details regarding the course.

Course No.	: CS U111
Course Title	: Computational Thinking & Programming
Instructor-in-charge	: Prof. Vinti Agarwal
Teaching Instructors	: Prof. Vinti Agarwal (vinti.agarwal@pilani.bits-pilani.ac.in) : Prof. Yash Sinha (yash.sinha@pilani.bits-pilani.ac.in) : Prof. Siddharth Mehrotra (siddharth.mehrotra@pilani.bits-pilani.ac.in) : Prof. Sundaresan Raman (sundaresan.raman@pilani.bits-pilani.ac.in)
Lab Instructors	: Prof. Jagat Sesh Challa (jagatsesh@pilani.bits-pilani.ac.in) : Prof. Rajesh Kumar (rajesh.k@pilani.bits-pilani.ac.in)

The List of PhD lab instructors with contact emails is provided below.

Manasvi Singh(RS) p20240481@pilani.bits-pilani.ac.in	Ashin Babu(RS) p20240495@pilani.bits-pilani.ac.in	Swetha V(RS) p20230083@pilani.bits-pilani.ac.in
Radhika Bohra(RS) p20240428@pilani.bits-pilani.ac.in	Ramakant Upadhyay(RS) p20220021@pilani.bits-pilani.ac.in	Ayushi Singh(RS) p20250409@pilani.bits-pilani.ac.in
Ravi Kumar Sharma(RS) p20240430@pilani.bits-pilani.ac.in	Esha Srivastava(RS) p20240902@pilani.bits-pilani.ac.in	Animesh Bhargava p20260098@pilani.bits-pilani.ac.in
Harsh Pandey p20260809@pilani.bits-pilani.ac.in	Mohit p20210472@pilani.bits-pilani.ac.in	MUKUL SAINI p20260025@pilani.bits-pilani.ac.in
ANKALP HARITASH p20260029@pilani.bits-pilani.ac.in		

1. Course Description:

Basic model of computation, input, processing, output, and storage; problem decomposition, abstraction, and representation of solutions using flowcharts and pseudocode. Algorithmic thinking, formulation of algorithms, tracing and verification of correctness, and comparison of efficiency at a conceptual level. Programming fundamentals, expressions, variables, primitive and composite data types, type conversion, and operator precedence; conditional statements and structured decision making; bounded and unbounded iteration. Modular program design, functions, parameter passing, scope, and structured program development. Recursion, recursive

problem solving, base and recursive cases, call stack behaviour, and comparison with iterative approaches. Data structures, arrays, strings, and structured data representations; pointers, dynamic memory allocation, and memory management. File handling, basic input/output operations and sequential data processing. AI-assisted programming, responsible use of AI tools for code generation, including prompt formulation, critical evaluation, debugging, and verification of generated code.

Laboratory: Algorithm design and tracing; implementation of programs involving conditionals, iteration, functions, recursion, arrays, strings, structured data, pointers, and dynamic memory; file handling; and integrated problem-solving exercises with critical evaluation of AI-assisted code.

1. Scope and Objective:

This course establishes computational problem solving as a discipline before it establishes programming as a skill. Students first learn to decompose a problem, abstract away detail, express a solution as an algorithm, and reason about its correctness and cost. They then learn to realise those solutions as working programs, progressing from control structures and modular design to recursion, structured data, pointers, and explicit memory management. The course is anchored in C so that students build an accurate mental model of storage, addressing, and program execution, which is the model assumed by subsequent courses in data structures, operating systems, computer architecture, and networks.

This subject aims to achieve the following goals:

- To develop the ability to decompose problems, abstract essential detail, and express solutions as algorithms independent of any programming language.
- To build fluency in structured programming, covering data types, expressions, control constructs, and modular function design.
- To establish an accurate mental model of program execution and memory through recursion, pointers, and dynamic memory management.
- To develop competence in representing and processing structured and linked data, and in persisting data through file operations.
- To build the judgement required to evaluate, test, and verify AI-generated code, and to use AI programming tools responsibly and with due attribution.

2. Course Learning Outcomes:

Upon successful completion of this course, students will be able to:

CLO1: Express computational problems using abstraction, algorithmic thinking, and structured representations including flowcharts and pseudocode.

CLO2: Implement programs using fundamental constructs including variables, data types, conditionals, iteration, and modular function design.

CLO3: Apply recursion and dynamic memory management to solve problems involving structured and linked data representations.

CLO4: Implement file handling operations for sequential data processing and structured input/output.

3. Textbooks and References:

T1. Jeri R. Hanly and Elliot B. Koffman, Problem Solving and Program Design in C, 8th Edition, Pearson, 2015.

T2. Brian W. Kernighan and Dennis M. Ritchie, The C Programming Language, 2nd Edition, Prentice Hall, 1988.

R1. Stephen G. Kochan, Programming in C, 4th Edition, Addison-Wesley Professional, 2014.

R2. Tom Taulli, AI-Assisted Programming: Better Planning, Coding, Testing, and Deployment, O'Reilly Media, 2024.

R3. Peter J. Denning and Matti Tedre, Computational Thinking, MIT Press Essential Knowledge Series, 2019.

R4. Behrouz A. Forouzan and Richard F. Gilberg, Computer Science: A Structured Programming Approach Using C, 3rd Edition, Cengage Learning, 2007.

R5. B. S. Gottfried and Jitender Chhabra, Programming with C, Schaum's Outlines Series, 3rd Edition, McGraw Hill Education, 2017.

R6. Internet resources, standard library documentation, and current documentation of AI-assisted development tools, as circulated by the instructor.

4. Course Plan:

Week	Lec.	Module	Lecture Topics	Ref.	Learning Outcome
1	3	Computational Thinking Foundations	Basic model of computation, input, processing, output and storage; problem decomposition, abstraction, pattern recognition. Introduction to Programming Languages.	T1: Ch1	CLO1: Frame a problem in computational terms
2	3	Algorithm Design and Representation	Algorithms: efficiency, stepwise refinement, algorithm tracing, verification; Pseudocode, Flowcharts using Mermaid	R4: Ch1, slides	CLO1: Represent and verify algorithmic solutions
3	3	Programming Fundamentals	Program structure, compilation and execution model, Numbering system, variables, data types, expressions, type conversion, operator precedence, formatted input and output. The programming toolchain, including AI code generation assistants, prompt formulation, and critical evaluation and verification of generated code.	T1: Ch2 R2: Ch1, Ch3, slides	CLO2: Write and execute basic structured programs
4	3	Selection and Structured Decision Making	Conditional statements, nested conditions, else-if chains, switch construct, conditional operator	T1: Ch4	CLO2: Implement decision logic correctly
5	3	Iteration	Counter-controlled and sentinel-controlled repetition, for, while	T1: Ch5	CLO1, CLO2: Design and reason about

Week	Lec.	Module	Lecture Topics	Ref.	Learning Outcome
			and do-while constructs, nested loops, loop termination and correctness		repetitive computation
6	3	Modular Program Design	Top-down design with functions, function declaration and definition, parameter passing by value, return values, function reuse	T1: Ch3	CLO2: Decompose programs into functions
7	3	Program Organisation, Scope and Debugging	Scope, storage classes and lifetime, personal libraries, header and implementation files, separate compilation, defensive programming and systematic debugging	T1: Ch12	CLO2: Organise and debug larger programs
8	3	Arrays	One-dimensional and multidimensional arrays, declaration and initialisation, traversal, arrays as function arguments, searching and sorting	T1: Ch7	CLO2: Process linear and tabular collections
9	3	Strings and Text Processing	Character data, strings as character arrays, null termination, string library functions, text parsing and manipulation	T1: Ch8	CLO2: Implement text processing solutions
10	3	Recursion	Recursive problem solving, base and recursive cases, call stack behaviour, tracing recursive calls, recursion compared with iteration	T1: Ch9	CLO3: Formulate and trace recursive solutions
11	3	Pointers	Address and dereference operators, pointer declaration, pointer arithmetic, pointers with arrays, pointers as function parameters, output parameters	T1: Ch6	CLO3: Use indirection and reference semantics
12	3	Structured Data	Structures, member access, nested structures, arrays of structures, unions, enumerations, pointers to structures	T1: Ch10	CLO3: Represent composite records
13	3	Dynamic Memory and Linked Representations	Dynamic allocation and release, heap and stack storage, memory leaks, dangling pointers, linked list representation and traversal	T1: Ch13	CLO3: Manage run-time memory and linked structures

Week	Lec.	Module	Lecture Topics	Ref.	Learning Outcome
14	2	File Handling	Streams and file modes, opening and closing files, sequential text input and output, formatted and record-oriented processing, end-of-file detection and error handling	T1: Ch11	CLO4: Persist and retrieve data through files

5. Laboratory Plan: All labs starting from Lab 4 - Lab 13 will be evaluative. Two buffer lab sessions are included in the schedule to accommodate up to two absences, therefore the best 8 of 10 evaluative labs will be considered for grading.

Note: AI tools may be used for the laboratory sessions only as permitted in the lab sheet.

1. Introduction to Operating System (Unix), Basic Commands (*Non-evaluative*)
2. Writing basic C programs, compiling and executing them (*Non-evaluative*)
3. Data types, expressions, type conversion, formatted input and output (*Non-evaluative*)
4. Hand simulation of algorithms via Flowchart design using Mermaid CLI. (6 marks -**Mandatory**)
5. Conditional logic, nested conditions, controlled loops and nested loops, termination analysis (3 marks)
6. Function decomposition of a multi-part problem, parameter passing, function reuse (3 marks)
7. Multi-file program organisation, header files, systematic debugging exercise (3 marks)
8. One and two-dimensional array processing, search and sort implementations (3 marks)
9. String processing, text parsing and manipulation problems (3 marks)
10. Recursive implementations with call-stack tracing, recursive versus iterative comparison (3 marks)
11. Pointer arithmetic, call by reference, structures and arrays of structures (3 marks)
12. Dynamic memory allocation and release, leak identification (3 marks)
13. Linked lists- construction, traversal, Node addition/deletion (3 marks)
14. Sequential file R/W, record processing, error handling, Command line arguments. (3 marks)

7. Evaluation Scheme

Components	Duration	Marks	Data and Time	Nature of component (Closed Book/ Open Book)
Mid-Semester Test	90 min	50	Oct 08 AN2	Closed Book
Comprehensive Examination	3 hrs	100	Dec 02 AN	* Open/Closed Book
Programming Quiz	1-2 hrs	70	Nov 15	Open-book (Limited to material uploaded on the exam portal). Students are not allowed to bring anything with them.
Lab participation	2 hrs every week	30 (Best 8 out of	-	Continuous assessment†

		10)		
Poster presentation	7-8 mins per group)	50	Oct 24 & Oct 25	Group Presentation

*** Textbooks, Printout of Lecture slides, Printout of Textbooks (chapter-wise spiral binding), Handwritten notes (in Notebooks only, NO LOOSE PAPER NOTES)**

NC Criteria: NC will be awarded if the total score out of 300 is 30 or less.

8. Consultation Hours: We encourage students to meet during consultation hours only with the concerned faculties. However, You can also contact us with a prior appointment via email.

Prof. Vinti Agarwal: F6 (6121D)

Prof. Yash Sinha: T9

Prof. Siddarth Mehrotra: W3

Prof. Sundaresan Raman: Th8

Prof. Jagat Sesh Challa: M9

Prof. Rajesh Kumar: T9

9. Notices: All announcements will be done through the Nalanda LMS.

10. Make-up Policy: Make-ups are generally considered only for medical emergencies leading to hospitalization (or a personal emergency of similar nature). The decision by AUGSD and the Instructor-in-Charge regarding granting make-ups shall be final. Makeups are allowed for Mid-Semester Test, Comprehensive Examination and Programming Test. **No makeup for weekly laboratory sessions.**

11. Note: Award of grades would be guided in general by the histogram of marks. If a student does not give sufficient opportunity for being assessed, either by missing a component entirely or by not applying oneself to the task seriously, he/she may be awarded an 'NC' report.

12. Disciplinary Policy: If we find you using unfair means during any of the evaluative components, the case will be directly reported to AUGSD and shall be dealt with very seriously as per the guidelines. You can find the AUGSD guidelines for unfair means cases over here

https://academic.bits-pilani.ac.in/Downloads/Unfair_Means_Offence_Revised_02_AUG_2022.pdf

**Instructor-in-charge
CS U111**